

DevOps Lead Interview Questions

Part 2 - Advanced Technical · 50 Questions · 5 Tool-Focused Topics

Argo CD (10)

GitHub (10)

CloudBees (10)

Runners (10)

EKS (10)

Argo CD

Q1. What is Argo CD and what core problem does it solve?

ANSWER

Argo CD is a declarative GitOps continuous delivery tool for Kubernetes. It continuously monitors Git repositories containing Kubernetes manifests and reconciles the live cluster state to match. It solves drift between desired and actual state, provides a clear audit trail (Git history = deployment history), enables easy rollbacks (git revert), and shifts deploys from imperative push to declarative pull.

Q2. Explain the Argo CD Application and ApplicationSet resources.

ANSWER

An Application is a CRD that defines: source (Git repo, path, revision), destination (cluster, namespace), and sync policy. It's the unit Argo CD reconciles. ApplicationSet is a controller that generates multiple Applications from templates using generators (list, cluster, git, matrix, pull request). Use ApplicationSet to deploy the same app to many clusters or to create preview environments per PR.

Q3. What are sync waves and sync hooks in Argo CD?

ANSWER

Sync waves order resource application using the annotation 'argocd.argoproj.io/sync-wave: -1' (lower waves apply first). Useful for CRDs before CRs, or DBs before apps. Sync hooks run at lifecycle points: PreSync (before sync, e.g. DB migration job), Sync, PostSync (smoke tests), SyncFail (cleanup). Hooks are regular K8s resources annotated with 'argocd.argoproj.io/hook'.

Q4. How do you handle secrets in an Argo CD GitOps workflow?

ANSWER

Never commit plain secrets to Git. Options: (1) Sealed Secrets (Bitnami) — encrypt with cluster public key, only the controller can decrypt. (2) External Secrets Operator — sync from Vault, AWS Secrets Manager, etc. (3) SOPS + KSOPS plugin — encrypted YAML with KMS. (4) Argo CD Vault Plugin — replace placeholders at sync time. ESO is the modern preferred approach.

Q5. What is the Argo CD App of Apps pattern?

ANSWER

A single root Application points to a Git path containing other Application manifests. Syncing the root creates and manages all child apps. The pattern enables bootstrapping a new cluster with one apply, hierarchical organization (platform apps then team apps then workloads), and managing many apps without manually creating each. ApplicationSet has largely superseded this for dynamic generation.

Q6. Explain the difference between auto-sync, self-heal, and prune.

ANSWER

Auto-sync: when Git changes, automatically apply to cluster (default is manual). Self-heal: if someone changes a resource directly in-cluster (kubectl edit), Argo CD reverts it back to Git state — prevents drift. Prune: when a resource is removed from Git, delete it from the cluster (off by default to prevent accidental deletion). Enable all three for true GitOps; carefully consider prune for prod.

Q7. How does Argo CD compare to Flux?

ANSWER

Argo CD has a strong web UI, better visualization, and simpler mental model — one app equals one Git path. Flux is more modular (source-controller, kustomize-controller, helm-controller separately), more CLI/GitOps-pure, and pioneered the OCI artifact source model. Both implement GitOps; choice often comes down to whether the team values UI (Argo) or composable controllers (Flux). Both are CNCF graduated.

Q8. What strategies do you use for promoting changes across environments?

ANSWER

Common patterns: (1) Branch-per-env (dev branch to dev cluster, main to prod) — simple but creates merge overhead. (2) Folder-per-env in one repo with Kustomize overlays — promote by PR copying values between folders. (3) Separate config repo with image tags written by CI — promote by updating tag in the next env's file. (4) Argo CD Image Updater for auto-promotion based on image tag patterns. Folder + Kustomize is the most common.

Q9. How do you debug an Application stuck in OutOfSync or Degraded?

ANSWER

Use the UI App Diff to see exactly which fields differ between Git and cluster — often a mutating webhook or defaulting issue. Check Events tab for sync errors. Run 'argocd app get NAME' for details. For Degraded: check pod health, image pull errors, failing health checks. Common causes: missing CRDs (use sync waves), webhook injecting fields (use ignoreDifferences), or RBAC denying the controller.

Q10. How would you scale Argo CD to manage hundreds of clusters?

ANSWER

Run multiple Argo CD instances (per region/team) rather than one giant one. Shard the application-controller across replicas using cluster sharding. Use ApplicationSet with cluster generator for dynamic management. Enable Redis HA. Use repo-server replicas for parallel manifest generation. Consider Argo CD's high-availability manifests. Monitor app-controller queue depth — that's your scaling signal.

GitHub

Q11. Compare GitHub Actions to Jenkins for CI/CD.

ANSWER

GitHub Actions is YAML-based, tightly integrated with the repo (no separate server), uses ephemeral runners, has a marketplace of reusable actions, and bills by minutes. Jenkins is self-hosted, Groovy/declarative pipelines, requires infrastructure management, has 1800+ plugins, and gives total control. Actions wins for repo-native workflows; Jenkins wins for complex orchestrations and on-prem requirements.

Q12. Explain reusable workflows vs composite actions.

ANSWER

A reusable workflow is an entire workflow file (jobs, steps) called from another with 'uses: org/repo/.github/workflows/file.yml@ref' — runs in its own job context. A composite action bundles multiple steps into a single action you call inside one job's steps. Use composite actions for reusable step sequences (setup, build); reusable workflows for entire pipeline templates shared across repos.

Q13. How do you implement OIDC authentication from GitHub Actions to AWS?

ANSWER

Create an OIDC identity provider in AWS IAM pointing at token.actions.githubusercontent.com. Create an IAM role with a trust policy restricting which repo/branch/environment can assume it (sub claim like 'repo:org/repo:ref:refs/heads/main'). In the workflow, use 'aws-actions/configure-aws-credentials' with role-to-assume. No long-lived AWS keys in GitHub secrets — short-lived STS tokens minted per run.

Q14. What are GitHub Environments and why use them?

ANSWER

Environments (staging, prod) are deployment targets with: required reviewers (manual approval gates), wait timers, deployment branch restrictions, and environment-scoped secrets. Workflows reference them with 'environment: production'. They give you compliance-friendly approval flows and scope sensitive credentials so dev can't access prod secrets. Combine with OIDC for environment-bound IAM roles.

Q15. How do you secure GitHub Actions against supply chain attacks?

ANSWER

(1) Pin actions to a full SHA, not a tag (tags can be moved). (2) Use 'permissions:' to scope GITHUB_TOKEN to minimum (read-only by default). (3) Restrict which actions can run via org policies (allowed actions list). (4) Use Dependabot to track action updates. (5) Avoid pull_request_target unless you understand the risk — it runs with secrets on PR code. (6) Audit third-party actions before adopting.

Q16. Explain branch protection rules you would set on main.

ANSWER

Require PRs (no direct push), require N approving reviews, dismiss stale approvals on new commits, require status checks to pass (CI, tests, security scans), require branches to be up-to-date, require signed commits, require linear history, restrict who can push, and disable force push. Add CODEOWNERS for required reviewers on specific paths. Apply to release branches too.

Q17. What is the difference between GITHUB_TOKEN and a Personal Access Token?

ANSWER

GITHUB_TOKEN is automatically created per workflow run, scoped to the repo, expires when the job ends, and has permissions you can scope via the 'permissions:' key. PAT is user-owned, broad-scope, long-lived, and ties workflows to a personal account (a leaver problem). Use GITHUB_TOKEN by default; for cross-repo operations use a GitHub App token, not a PAT.

Q18. How do you implement matrix builds and what are their limits?

ANSWER

Define 'strategy.matrix' with arrays of values; GitHub generates a job per combination. Use 'include' to add specific extra combos, 'exclude' to drop combinations, and 'fail-fast: false' to let all jobs finish on failure. Limit is 256 jobs per matrix workflow run. For dynamic matrices, generate the matrix JSON in a setup job and pass via 'needs.setup.outputs.matrix' with fromJSON().

Q19. What are GitHub Apps and when would you use one?

ANSWER

GitHub Apps are first-class integrations with their own identity, fine-grained permissions, and short-lived installation tokens. Use one when: you need to act across multiple repos in an org without tying actions to a user (no PAT), require higher API rate limits (15,000/hr vs 5,000), need webhook event handling, or build internal tooling. Apps avoid the leaver problem PATs create.

Q20. How do you optimize a slow GitHub Actions workflow?

ANSWER

Cache dependencies with actions/cache (key on lockfile hash). Parallelize with matrix and split test suites. Use larger runners for CPU-bound jobs. Skip jobs with path filters or 'if:' conditions. Use job-level 'needs' for dependency graphs, not sequential steps. Pre-build Docker images and reuse via the registry cache. Run only changed services on monorepos (use dorny/paths-filter).

CloudBees

Q21. What is CloudBees CI and how does it relate to Jenkins?

ANSWER

CloudBees CI is the commercial, enterprise-grade distribution of Jenkins. It adds: Operations Center (manage many controllers from one place), high-availability mode, role-based access control with folders, validated plugins, support for Kubernetes-native controller provisioning (Managed Masters), and enterprise support. Same Jenkins pipeline DSL works on both.

Q22. Explain CloudBees Operations Center.

ANSWER

Operations Center is a central management plane that orchestrates multiple Jenkins controllers (called 'Managed Controllers' or 'Client Controllers'). It provides shared agents/cloud configs, cross-controller analytics, shared credentials, security realm propagation, and team isolation (each team gets a controller, but admin is centralized). Solves the scale problem of running one giant Jenkins.

Q23. How do you structure pipelines in CloudBees for many teams?

ANSWER

Use folders for team boundaries with folder-level credentials and RBAC. Put pipeline logic in shared libraries (Global Pipeline Libraries) versioned in Git, loaded with @Library. Provide standard pipeline templates so teams compose rather than copy-paste. Use Multi-Branch Pipelines for branch-per-feature workflows. Enforce standards with Pipeline Policies (CloudBees feature).

Q24. What are Jenkins Shared Libraries and how do they work?

ANSWER

Shared Libraries are Groovy code in a Git repo (vars/, src/, resources/) loaded into pipelines via @Library('name@version'). 'vars/' holds DSL-style steps (foo.groovy = call() method). 'src/' holds classes. They let you DRY up pipeline logic across hundreds of jobs. Version them via Git tags/branches; pin in pipelines for stability. Test with Jenkins Pipeline Unit framework.

Q25. How do you handle credentials securely in CloudBees pipelines?

ANSWER

Use Jenkins Credentials store (encrypted by master.key). Reference in pipelines with 'withCredentials' or 'credentials()' binding — never echo or log them. Scope credentials to folders/teams for least privilege. For cloud auth, prefer IAM instance roles or OIDC federation over static keys. Integrate HashiCorp Vault via the Jenkins Vault plugin for centralized rotation. Audit credential usage via CloudBees CI Reports.

Q26. Explain Jenkins agents (formerly slaves) and provisioning strategies.

ANSWER

Agents execute build workloads; the controller orchestrates. Strategies: (1) Static agents — long-lived VMs, simple but waste idle capacity. (2) Cloud agents — provisioned on-demand from AWS/Azure/GCP, terminated after the build. (3) Kubernetes agents — pods spawned per build using the Kubernetes plugin, fastest and most cost-efficient. K8s agents are the modern default; define agent pod specs in the Jenkinsfile.

Q27. What is a Multi-Branch Pipeline?

ANSWER

A job type that automatically discovers branches and PRs in a Git repo, creating a sub-job per branch. Each runs the Jenkinsfile from that branch. Branches removed in Git are auto-cleaned (per retention policy). Enables PR-based workflows: every PR builds with its own Jenkinsfile, status posted back to GitHub/Bitbucket. Pair with branch indexing webhooks for instant triggers.

Q28. How do you upgrade Jenkins/CloudBees CI safely in production?

ANSWER

Test on staging first with prod-equivalent plugin set. Take a config backup (ThinBackup or filesystem snapshot of JENKINS_HOME). Check plugin compatibility against the new core version with the Plugin Manager. Read the upgrade guide for breaking changes. Run rolling controller upgrades via Operations Center. Have a documented rollback plan. CloudBees Assurance Program (CAP) validates plugin sets.

Q29. How do you implement RBAC in CloudBees?

ANSWER

Use Role-Based Access Control plugin (CloudBees enhanced). Define roles globally (admin, developer, viewer) and at folder/project level. Bind groups (LDAP/SSO) to roles. Use folder-based RBAC so team A can't see team B's jobs or credentials. Operations Center can propagate roles across all controllers. Audit access with CloudBees Audit Trail plugin. Avoid using anonymous read for sensitive setups.

Q30. How do you monitor Jenkins/CloudBees CI health in production?

ANSWER

Expose Prometheus metrics via the Prometheus plugin (build duration, queue length, executor utilization, plugin update health). Alert on long queue times, frequent build failures, high disk usage on JENKINS_HOME, and controller heap pressure. Use CloudBees CI Reports for build trends. Log aggregation to ELK/Splunk. Track CI throughput (DORA metrics) and time-to-feedback on PRs.

Runners

Q31. What are GitHub Actions self-hosted runners and why use them?

ANSWER

Self-hosted runners are machines you provide to execute GitHub Actions jobs instead of GitHub-hosted ones. Reasons to use: access to private network resources (internal artifact repos, on-prem services), larger/specialized hardware (GPUs, big RAM), longer job times, custom OS/tooling pre-installed, and cost control at high usage volumes. Tradeoff: you manage scaling, patching, and security.

Q32. How do you securely run self-hosted runners?

ANSWER

Never use self-hosted runners on public repos — forked PRs could run malicious code with your secrets. Run runners in ephemeral mode (`--ephemeral` flag, one job then exit) so state doesn't persist between jobs. Isolate per-team using runner groups. Apply least-privilege IAM (no broad roles on the host). Patch the OS regularly. Use organization-level runners with restricted repo access, not enterprise-wide.

Q33. What is Actions Runner Controller (ARC) for Kubernetes?

ANSWER

ARC is a Kubernetes operator that provisions ephemeral self-hosted runners as pods on demand. You declare a `RunnerScaleSet` CRD with min/max replicas; ARC scales runners based on the GitHub Actions job queue (autoscaling on pending workflow runs). Each runner pod is created for one job then deleted — clean, secure, no state leakage. The modern recommended approach for self-hosted at scale.

Q34. How do you autoscale self-hosted runners?

ANSWER

Use Actions Runner Controller on Kubernetes with `HorizontalRunnerAutoscaler` (legacy) or `RunnerScaleSet` (new). Scale based on `workflow_job` webhooks (queue depth) or pull-based polling. Set min replicas for instant capacity, max to cap cost. On AWS without K8s, use ASGs with the `actions-runner-system` project or Philips Labs Terraform module which spins EC2 spot instances per queued job.

Q35. Compare GitHub-hosted vs self-hosted runners on cost.

ANSWER

GitHub-hosted: ~\$0.008/min for Linux (free for public repos, 2000 free minutes/month on Pro). Simple, no maintenance, but Linux 4 vCPU is the standard. Self-hosted: pay for your own compute (EC2, K8s nodes) plus ops time. Break-even depends on usage — heavy users (over 50k mins/month) typically save with self-hosted spot instances; light users should stay on GitHub-hosted for the operational simplicity.

Q36. What is a Jenkins Kubernetes agent and how does it work?

ANSWER

The Jenkins Kubernetes plugin creates pods on a K8s cluster as build agents. You define a pod template in the Jenkinsfile with 'agent { kubernetes { yml ... } }', specifying containers (build tools, sidecars). Jenkins controller spawns the pod per build, runs steps, then deletes the pod. Each build gets a clean environment, no agent maintenance, near-zero idle cost — and it autoscales naturally with cluster autoscaler.

Q37. How do you debug a self-hosted runner that won't pick up jobs?

ANSWER

Check runner status in repo/org Settings to Actions to Runners (Idle/Active/Offline). On the host, inspect `_diag/` logs and `Runner_*.log`. Verify labels match the workflow's runs-on. Check network connectivity to github.com (HTTPS 443). For ARC, kubectl logs the runner pod and controller. Restart with `svc.sh stop/start`. Token expired? Re-register. Cache full? Clear `_work/`.

Q38. How do you isolate workloads on shared self-hosted runners?

ANSWER

Best practice: don't share — use ephemeral runners (one job per runner) so no state crosses jobs. If you must share, run jobs in containers (Docker-in-Docker or container jobs), use per-job working directories, never run as root, and clean caches/secrets aggressively. Use runner groups to scope which repos can access which runners. K8s with ARC handles isolation correctly out of the box.

Q39. What are the trade-offs of running runners as Docker containers vs VMs?

ANSWER

Containers: fast startup (seconds), low overhead, easy to scale on K8s, but weaker isolation — privileged builds (DinD) widen attack surface. VMs: stronger isolation, can run any kernel feature, but slower to provision and more expensive. Use containers (K8s ARC) for typical CI workloads; VMs for builds requiring kernel modules, custom drivers, or stricter security (think regulated industries).

Q40. How do you handle artifact caching across ephemeral runners?

ANSWER

Don't rely on host disk — ephemeral runners are wiped. Use: (1) GitHub Actions cache (actions/cache) with key strategy on lockfile hash. (2) Remote build caches like sccache (Rust), Gradle Build Cache, Bazel Remote Cache, BuildKit registry cache for Docker. (3) Artifact store (S3, Artifactory) for built outputs. Tune cache hit rates by using stable cache keys; partial hits via 'restore-keys' fallback.

EKS

Q41. Difference between EKS managed node groups, self-managed nodes, and Fargate?

ANSWER

Managed node groups: AWS provisions EC2 nodes via ASGs; AWS handles AMI updates with drain/replace; you choose instance types. Self-managed: you create the ASG and AMIs yourself — more flexibility (custom OS, bottlerocket variants), more ops. Fargate: serverless pods — no nodes to manage, billed per pod per second; tradeoffs are limited DaemonSet support, no GPU, higher per-pod cost, and slower pod start.

Q42. Explain IAM Roles for Service Accounts (IRSA) in EKS.

ANSWER

IRSA lets a pod assume an IAM role via its ServiceAccount. EKS publishes an OIDC provider; you create an IAM role with a trust policy on that OIDC issuer and the specific namespace/SA. Annotate the SA with the role ARN. Pods get short-lived STS credentials injected automatically. Eliminates static AWS keys in pods and gives per-workload least-privilege. Modern alternative: EKS Pod Identity (simpler, no OIDC needed).

Q43. How do you scale an EKS cluster?

ANSWER

Two layers: (1) Pod-level — Horizontal Pod Autoscaler scales replicas on CPU/memory/custom metrics; Vertical Pod Autoscaler adjusts requests. (2) Node-level — Cluster Autoscaler watches for unschedulable pods and scales the ASG, or use Karpenter which provisions right-sized nodes directly without ASGs (faster, more efficient bin-packing). Karpenter is the modern preference for production EKS.

Q44. What is Karpenter and how does it differ from Cluster Autoscaler?

ANSWER

Karpenter is an open-source node provisioner (originally AWS). It watches pending pods and launches EC2 instances directly that fit the workload's exact requirements — no preconfigured ASGs needed. Faster (under a minute vs several), more flexible (mixes instance types, picks Spot), and consolidates underused nodes automatically. Cluster Autoscaler is tied to per-instance-type ASGs; Karpenter is workload-aware.

Q45. How do you handle networking in EKS — VPC CNI, IP exhaustion, custom CNI?

ANSWER

AWS VPC CNI is default — each pod gets a real VPC IP from ENIs attached to the node. Risk: IP exhaustion (subnets too small for pod density). Mitigations: prefix delegation (more IPs per ENI), custom networking (pods on a secondary CIDR), or switch to Cilium/Calico with overlay networking. Plan subnets with at least 2x pod count per AZ. Reserve /20 or bigger subnets for production.

Q46. How do you implement zero-downtime deploys on EKS?

ANSWER

Use Deployment with RollingUpdate strategy (maxSurge/maxUnavailable tuned). Configure readinessProbe so traffic only routes to ready pods. Set 'terminationGracePeriodSeconds' high enough for inflight requests to finish, and 'preStop' hook to deregister from LB before SIGTERM. Use PodDisruptionBudget to prevent voluntary evictions taking down too many replicas. For Service of type LoadBalancer use NLB target-type IP with deregistration delay tuned.

Q47. Compare AWS Load Balancer Controller vs the in-tree cloud provider.

ANSWER

The in-tree provider (legacy) creates Classic ELBs from Service type LoadBalancer with limited config. AWS Load Balancer Controller is the modern approach: creates ALBs from Ingress (layer 7, host/path routing, WAF, OIDC auth) and NLBs from Service (layer 4, IP target-type so pods are direct targets — better performance). Install it on all production clusters; in-tree is deprecated.

Q48. How do you secure an EKS cluster?

ANSWER

(1) Private API endpoint or restrict public CIDR access. (2) IRSA/Pod Identity instead of static keys. (3) Network policies (Cilium/Calico) for pod-to-pod restrictions. (4) Use OPA Gatekeeper or Kyverno for admission policies (no privileged pods, image registry allowlist). (5) Encrypt secrets at rest with KMS. (6) Audit logs to CloudWatch with anomaly detection. (7) Image scanning on ECR. (8) Patch nodes regularly via managed node group updates.

Q49. How do you handle persistent storage on EKS?

ANSWER

Use the EBS CSI driver (block storage, single-AZ, single-pod RWO) for databases and stateful workloads. Use EFS CSI driver (NFS, multi-AZ, RWX) for shared file storage across pods. FSx for Lustre for HPC-style workloads. Define StorageClasses with reclaimPolicy and volumeBindingMode 'WaitForFirstConsumer' so volumes are created in the right AZ. Snapshot via VolumeSnapshot CRDs for backups.

Q50. Walk through how you would design an EKS deployment for a critical production app.

ANSWER

Multi-AZ EKS cluster across 3 AZs, private API endpoint, Karpenter for node provisioning with Spot+On-Demand mix. AWS LB Controller fronts traffic via ALB with WAF. IRSA/Pod Identity for AWS access, External Secrets Operator pulling from Secrets Manager. Argo CD for GitOps deploys with auto-sync + self-heal disabled in prod (manual approval gate). HPA + PDB on each Deployment. Prometheus + Grafana + Loki for observability, with alerts on golden signals tied to SLOs. CloudWatch Container Insights as backup. Velero for cluster backup. Documented runbooks and regular DR drills.